



SÃO PAULO | 03 DE SETEMBRO DE 2026

AWS SUMMIT



DEV203

Dia de faxina nos custos!

Automação FinOps com policy-as-code

Felipe KiKo

Ele/Dele

Arquiteto Cloud

Avantpro

Não é novidade, mas o que é **FinOps?**

Cultura Colaborativa

FinOps **une** times de engenharia, finanças e negócios para decisões de custo com **responsabilidade compartilhada**

Visibilidade primeiro

Saber quanto custa, quem criou aquele recurso e se ainda faz sentido existir
Spoiler: nem sempre faz!

Recomendações AWS

AWS Cost Explorer, AWS Trusted Advisor, AWS Compute Optimizer...a AWS já te fala o que fazer, mas alguém **precisa agir**

Ciclo sem fim

FinOps não é projeto com data de entrega, é um **ciclo** de informar, otimizar e operar, e que **nunca deve parar!**

O problema: Sem **automação** = ciclo sem fim!

Não é *One Shot*

Você faz a faxina hoje...semana que vem já tem recurso novo, acumulando poeira de novo

Tudo muda

Novos serviços, novas pessoas, produto evoluindo...o cenário de custos nunca para quieto (e que bom!)

Tem os esquecidinhos!

Aquele “*é só pra testar*” que ninguém desligou e tá lá rodando a meses, sem nenhuma utilização

Mas...e a IA?

Não podemos fazer tudo isso com ela?

Claro que sim! ...**porém**

- Mas e **consumo** de tokens / créditos da IA?
- É uma tarefa **repetitiva** e de fácil **automatização**? Mmm...será que compensa?
- Teremos garantia de ter sempre o mesmo **resultado**?
- E a **manutenção** “invisível” quando existem mudanças, por exemplo de API?
- Não estamos usando uma **bazuca** para matar barata?
- E se talvez um **agente** de IA rodando um **motor** por trás dele...quem sabe?

E quem poderá nos ajudar?



EIS QUE APRESENTO...

Cloud Custodian

Conhecido também como C7N!

- Open Source!
- Policy as code (YAML)
- Suporte Multi Cloud e K8s
- Comunidade (Slack)



E o que ele faz?

Identifica → Notifica → Remedia

Faz o ciclo completo, partindo para a identificação, e com os resultados em mãos pode notificar e até remediar!

Quando usar?

Podemos usar para **FinOps**, **Segurança**, **Compliance** e até para **inventariados**, inclusive em todas as contas AWS (*c7n-org*)

100% serverless

Execução com AWS Lambda, sendo possível agendamento ou até mesmo real-time!

E o que é Policy-as-Code?

São **regras** escrita em **código** (YAML) que detecta e age automaticamente sobre recursos, sem a necessidade codificação ou scripts...similar ao funcionamento do Terraform, por exemplo!

Estrutura básica:

- Recurso: O que eu quero olhar?
- Filtro: Qual condição me preocupa?
- Ação: O que fazer quando encontrar?

Exemplos de uso (FinOps):

- EBS sem uso há 30 dias...**deletar**
- Instância DEV ligada fora do horário...**desligar**
- Snapshot com mais de 90 dias...**remover**
- IP elástico sem associação...**liberar**
- RDS em DEV com instância oversized...**notificar** o time

Exemplos de uso (além de custos):

- Bucket S3 público...**bloquear** acesso
- Security group com 0.0.0.0/0...**alertar**
- Recurso sem TAG obrigatória...**marcar** non-compliant



SHOW ME THE CODE!

Estrutura completa de uma Policy

```
1 policies:
2   - name: nome-da-policy           # Identificador único (slug)
3     resource: aws.ec2             # Tipo de recurso (350+ suportados)
4     description: |                # Descrição do controle
5       O que essa policy faz e por quê
6
7     mode:                          # Como executa (opcional - default: pull)
8       type: periodic              # periodic | cloudtrail | ec2-instance-state
9       schedule: rate(1 day)       # Frequência (cron ou rate)
10      role: arn:aws:iam::{account}:role/CustodianRole
11
12    filters:                        # Condições para selecionar recursos
13      - type: value                 # Filtro por atributo
14        key: AlgumaPropriedade
15        op: greater-than           # Operador
16        value: 30                  # Valor
17
18      - "tag:Environment": dev     # Filtro por tag (atalho)
19
20      - type: metrics               # Filtro por métrica CloudWatch
21        name: CPUUtilization
22        days: 14
23        value: 5
24        op: less-than
25
26    actions:                        # O que fazer com os recursos filtrados
27      - type: notify                # Notificar (via c7n-mailer)
28        template: alerta
29        to:
30          - resource-owner
31        transport:
32          type: sqs
33          queue: c7n-mailer
34
35      - type: mark-for-op           # Marcar para ação futura
36        tag: c7n-cleanup
37        op: stop
38        days: 7
39
40      - stop                        # Ação direta (string simples)
```



ALGUNS EXEMPLOS...

Faxina: EBS órfãos

```
1  policies:
2    - name: faxina-ebs-orfaos
3      resource: ebs
4      filters:
5        - Attachments: []
6        - type: value
7          key: CreateTime
8          value_type: age
9          value: 30
10         op: greater-than
11      actions:
12        - delete
```

ALGUNS EXEMPLOS...

Faxina: EC2 DEV fora do expediente

```
1  policies:
2    - name: faxina-ec2-offhours
3      resource: ec2
4      filters:
5        - type: offhour
6          tag: Schedule
7          default_tz: America/Sao_Paulo
8          offhour: 20
9      actions:
10     - stop
```



Faxina: RDS grande demais para ambiente DEV

```
1  policies:
2    - name: faxina-rds-oversized-dev
3      resource: rds
4      filters:
5        - "tag:Environment": dev
6        - type: value
7          key: DBInstanceClass
8          op: in
9          value:
10           - db.r5.xlarge
11           - db.r5.2xlarge
12           - db.r6g.xlarge
13           - db.m5.2xlarge
14      actions:
15        - type: notify
16          template: rds-oversized
17          to:
18            - resource-owner
19          transport:
20            type: sqs
21            queue: c7n-mailer
```

#FicaDica

--dryrun, seu melhor amigo!

Ele ajuda para que “**coisas não tão boas**” aconteçam em produção, mas também com **análise** e **entendimento** dos recursos!

mark-for-op, deixa as coisas para depois...

Dar o aviso antes de executar é sempre bom! Ajuda a controlar os recursos que insistem em não ficar em compliance

TAGs, abuse e use!

Usar TAGs para o **controle** das actions ajuda demais!
Exemplo: DoNotDelete

E o Kiro? Onde consegue nos ajudar?



Em diversos aspectos!

Estruturar

Criar nosso plano FinOps antes de tudo

Lembra? ***Spec-Driven!***

Codificar / Deployar

Gerar as policies, **validar**, **testar**, **atualizar**...aqui ele vai brilhar!
E depois ainda conseguir subir tudo para você!

Entender

Ele vai conseguir entender tudinho aquilo que foi **retornado!**

Documentar

Tanto o que você fez, como até qual é o jeito certo de fazer!
Ele gosta de documentações!

ELE PODE E DEVE SER...

Suíte completa de automações!

Centralizando tudo em um
único lugar!

IA + AWS + KIRO + C7N

- Ele pode **olhar** para o seu ambiente
 - Ver o que faz sentido **monitorar**
 - **Criar** suas policies e aplicar
 - **Entender** o retorno (talvez em um bucket S3?)
 - Ver se faz sentido **manter**, **mexer** ou **remover** a policy
 - **Documentar** e **notificar**
 - E...repetir!
-
- Inclusive se apoiando em **outras ferramentas!**

Comparativo Kiro VS AWS FinOps Agent (preview)

Minhas impressões em!

Kiro

- + Mais técnico
- + Consegue ir mais profundo
- + Pensa mais
- + Extrapolava somente os custos =)
- - Extrapolava somente os custos =(

AWS FinOps Agent

- + User Friendly
- + Automações / Integrações
- + Contexto / Artefatos
- + Guardrails
- - Centralizado
- - Menos autonomia



Legal! Mas...por onde começo?



Do Blueprint até a Governança!

Escolha o vilão

Veja quem está mais te incomodando...e ataque ele!

Comece pequeno

5 policies bem estruturadas são melhor do 50 esquecidas ou fracas

Automatize

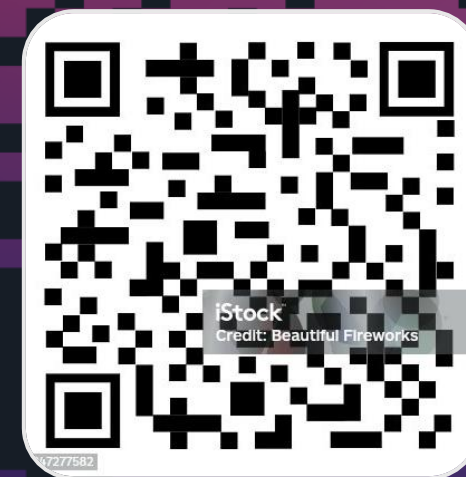
Tanto a criação como o deploy...não precisa criar um CI/CD completo no Day 1

Não esqueça

Implementar e não usar é ruim...veja o que ajudou e o que não ajudou para melhorar!



Agradecemos sua atenção



Preencha a pesquisa de satisfação
no aplicativo móvel